

Experiment Report
Brent Smith

Experiment 1

a.)

Algorithm	Input Type	# of Comparisons	Comments about Time Complexity and # Comparisons
Selection Sort	ordered.txt	49,995,000	Selection Sort is $O(n^2)$ in all cases, so this 8-digit comparison number aligns with the time complexity
Merge Sort	ordered.txt	69,008	5-digits, within the bounds of time complexity $O(n \log(n))$.
Heap Sort	ordered.txt	273,914	6-digits, within the bounds of time complexity $O(n \log(n))$. However, performs with larger comparison count relative to merge sort, implying much sort runs faster on larger arrays.
Quick Sort fp	ordered.txt	50,004,999	Performed at $O(n^2)$, dramatically worse than random pivot. An ordered list must be worst case for QuickSort when using the first element as the pivot point
Quick Sort rp	ordered.txt	152,029	6-digits, within the bounds of time complexity $O(n \log(n))$. Performed much better than first pivot on ordered list, so random pivot is a

			safer version to use to avoid worst case.
Selection Sort	random.txt	49,995,000	Selection sort runs the same time complexity in all cases, so it should be close to if not the exact same number of comparisons regardless of list orientation
Merge Sort	random.txt	181,624	6-digits, within the bounds of time complexity $O(n\log(n))$.
Heap Sort	random.txt	258,470	6-digits, within the bounds of time complexity $O(n\log(n))$. However, performs with larger comparison count relative to merge sort, implying much sort runs faster on larger arrays.
Quick Sort fp	random.txt	162,825	6-digits, within the bounds of time complexity $O(n\log(n))$. Performed similar to random pivot, showing that the first pivot average case is random arrays.
Quick Sort rp	random.txt	152,708	6-digits, within the bounds of time complexity $O(n\log(n))$. Performed close to first pivot.
Selection Sort	reverse.txt	49,995,000	Selection sort runs the same time complexity in all cases, so it should be

			close to if not the exact same number of comparisons regardless of list orientation
Merge Sort	reverse.txt	64,608	5-digits, within the bounds of time complexity $O(n\log(n))$.
Heap Sort	reverse.txt	243,394	6-digits, within the bounds of time complexity $O(n\log(n))$. However, performs with larger comparison count relative to merge sort, implying much sort runs faster on larger arrays.
Quick Sort fp	reverse.txt	50,004,999	Worst case time complexity is $O(n^2)$, and using the first index as the pivot point on a reverse order list is also the worst case. Therefore our 8-digit comparison being close to selection sort would match an $O(n^2)$ time complexity
Quick Sort rp	reverse.txt	150,458	6-digits, within the bounds of time complexity $O(n\log(n))$. Performed much better than first pivot on a reverse ordered list, so random pivot is a safer version to use to avoid hitting the worst case.

b.) The only time I used extra memory space was to create a private class variable int array to store the array for Merge Sort post-sorting. This was because my MergeSort sort method

recursively called itself and I had no means of returning the array otherwise. I did not use any other data structures.

c.)Based on my experimental results, MergeSort worked the best overall. It beat heap sort and both quicksorts in terms of number of comparisons. Comparing random pivot and first pivot QuickSort, random pivot is better because it is less probable that the orientation of the list would result in the worst case for QuickSort rp (safer), where the list just has to have some ordering to mess with QuickSort fp.

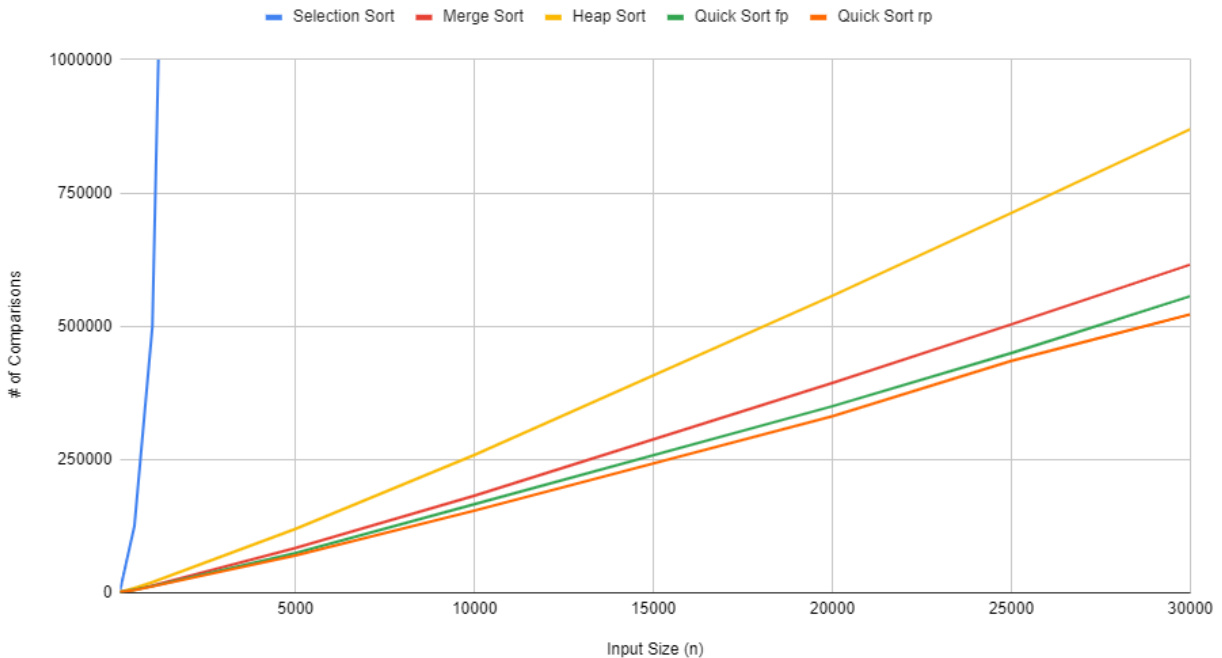
Experiment 2

Results Table: Average Numbers From Multiple Samples

	100	500	1000	5000	10000	20000	25000	30000
Selection Sort	4,950	124,750	499,500	12,497,500	49,995,000	199,990,000	312,487,500	449,985,000
Merge Sort	820	5,760	13075	83371	181520	393429	503287	615,323
Heap Sort	1,256	8,556	19158	119218	258277	556734	712424	869690
Quick Sort fp	740	4,987	11864	73900	165890	349689	449187	556169
Quick Sort rp	608	4,827	11322	69373	153836	331022	434723	521774

Results Chart

Comparison Count vs Input Size



Looking at the table and graphs, all of the functions seem to follow closely to their theoretical results. Given that SelectionSort has time complexity $O(n^2)$ it follows pretty closely within the digit range to qualify under that time complexity. All of the other functions with average case $O(n \log(n))$ stay within the digit bound of their complexity as well. MergeSort and both QuickSort's stay relative near each other while following their $O(n \log(n))$ pathway, and HeapSort had a noticeably larger number of comparisons to the other $O(n \log(n))$ functions. This is because the HeapSort function is an unstable function, and while having the time complexity similar to these "divide and conquer" sets, it functions much similar to selection sort. It begins to slow down as the input size gets larger. The reason for the inconsistencies between the experimental plot and theoretical plot is that a limited number of samples were used and the arrays generated were randomized between each sample, which allows the outliers to skew the average. By Law of Big Numbers, as the sample size approaches infinity (assuming proper sampling), the distribution smoothes out and the expected value converges to the theoretical value. If I were to take more samples, the average would more closely align with the theoretical.